

AhmedML Dataset Splits

Deterministic train/validation/test splits for the [AhmedML](#) dataset [1]. The split manifest is stored at `splits/manifest.json` as a flat JSON object with keys named `{split_name}_{train,val,test}` and case IDs that match the on-disk run directories.

AhmedML contains 500 geometric variations of the Ahmed car body. The public force/moment table contains all 500 runs. This split package intentionally does not store the large per-run STL or PNG assets. The committed lightweight artifacts are the manifest, aggregate CSVs, derived Chamfer/image metrics, and diagnostic figures. The geometry split is generated from STL-surface Chamfer scores in `data/chamfer_metrics.csv`. The image-wake split is generated from `UxMean` Y-4 centreline and near-base X-slice PNG scores in `data/image_metrics.csv`. The geometry-parameter table contains 499 rows; run 500 is absent from that file, so the generator mean-imputes its geometry parameters only for the nested data-efficiency ordering and records this in `data/parameter_geometry_metrics.csv`.

Splits at a glance

Split	Type	Train	Val	Test	What it tests
<code>full</code>	In-dist	400	50	50	Noether-compatible seed-42 random baseline split
<code>medium</code>	In-dist	133	50	50	Data efficiency, 1/3 of <code>full</code> training data
<code>scarce</code>	In-dist	67	50	50	Data efficiency, 1/6 of <code>full</code> training data
<code>super_scarce</code>	In-dist	11	50	50	Extreme data efficiency, 1/36 of <code>full</code> training data
<code>geometry</code>	OOD	350	50	100	STL-shape extrapolation using the top 20% local Chamfer-isolation score
<code>high_drag</code>	OOD	350	50	100	High-drag extrapolation using the top 20% <code>cd</code>
<code>low_drag</code>	OOD	350	50	100	Low-drag extrapolation using the bottom 20% <code>cd</code>
<code>image_wake</code>	OOD	350	50	100	Image-derived wake extrapolation using the top 20% <code>UxMean</code> Y-4 plus near-base X-slice score

Difficulty ladders:

- Data efficiency: `full` < `medium` < `scarce` < `super_scarce`, with fixed validation/test sets.
- OOD regimes: `geometry`, `high_drag`, `low_drag`, and `image_wake`.

Which split should I use?

- Simple baseline or literature compatibility: `full`
- Data efficiency study: compare `super_scarce`, `scarce`, `medium`, and `full`
- Geometry extrapolation: `geometry`
- Targeted coefficient extrapolation: `high_drag` or `low_drag`
- Image-derived wake extrapolation: `image_wake`

Using the committed splits

For standard benchmark use, consume the committed manifest at `splits/manifest.json`; no regeneration is required. The manifest is the source of truth for all split membership and contains one train, validation, and test key for each split listed above.

```
import json
from pathlib import Path

manifest = json.loads(Path("splits/manifest.json").read_text())

train_ids = manifest["geometry_train"]
val_ids = manifest["geometry_val"]
test_ids = manifest["geometry_test"]
```

Change the `geometry` prefix to `full`, `medium`, `scarce`, `super_scarce`, `high_drag`, `low_drag`, or `image_wake` to select a different split. Each value is a sorted list of `run_N` directory names that match the dataset case folders.

Design principles

- **Validation is always in-distribution with train.** For OOD splits, validation is drawn from the training-side population, not from the held-out extreme test region.
- **Keep the public baseline stable.** `full` matches Noether’s AhmedML seed-42 split: 400 train, 50 validation, and 50 test.
- **Use the source data for each regime.** Drag-regime splits use `force_mom_all.csv`; geometry-regime splits use externally downloaded STL files and `data/chamfer_metrics.csv`; image-regime splits use externally downloaded UxMean PNG files and `data/image_metrics.csv`.
- **Nested data-efficiency subsets.** `super_scarce_train` is a strict subset of `scarce_train`, `scarce_train` is a strict subset of `medium_train`, and `medium_train` is a strict subset of `full_train`.
- **Test set integrity.** Test cases should not be used for hyperparameter tuning, model selection, normalization fitting, or visual inspection-driven iteration.

Split details

full

The default public split is a seeded random split over all 500 run IDs, not a physics-stratified split. It constructs `torch.randperm(500)` with seed 42, shifts the IDs to 1..500, then assigns the first 400 IDs to train, the next 50 to validation, and the final 50 to test. The case IDs are sorted before being stored in the manifest. These IDs match the public [AhmedMLDefaultSplitIDs](#) implementation in Noether [3].

medium, scarce, and super_scarce

Same validation/test as `full`, but the training set is a nested force- and geometry-diverse subset of `full_train`: 133 cases for `medium`, 67 cases for `scarce`, and 11 cases for `super_scarce`. The subset order is a greedy max-min selection in standardized force/geometry feature space using `cd`, `cl`, and all available geometry parameters.

geometry

The geometry split is an STL-surface OOD split. Each `run_N/ahmed_N.stl` is sampled with 4096 deterministic area-weighted surface points. The sampled point clouds are compared using symmetric Chamfer RMS distance after scaling by the global median STL bounding-box diagonal. The geometry score is the mean Chamfer distance to the 10 nearest neighboring runs:

```
geometry_score_i = mean_10_nearest_neighbors(chamfer_distance_i)
```

The top 20% of runs by this local-isolation score form the OOD test set. Validation is sampled from the remaining training-side pool:

```
geometry_test = top_20_percent_i(geometry_score_i)
geometry_pool = all_runs - geometry_test
geometry_val = deterministic_random_sample(geometry_pool, round(0.125 * |geometry_pool|))
geometry_train = geometry_pool - geometry_val
```

In the current metrics, the most locally isolated STL cases include `run_52`, `run_153`, `run_244`, `run_480`, `run_410`, `run_96`, `run_431`, and `run_91`. Figure 3 shows the geometry split against the Chamfer score used to define the holdout. Figure 1 shows a low-score and high-score geometry example, with both STLs projected in the same coordinate frame.

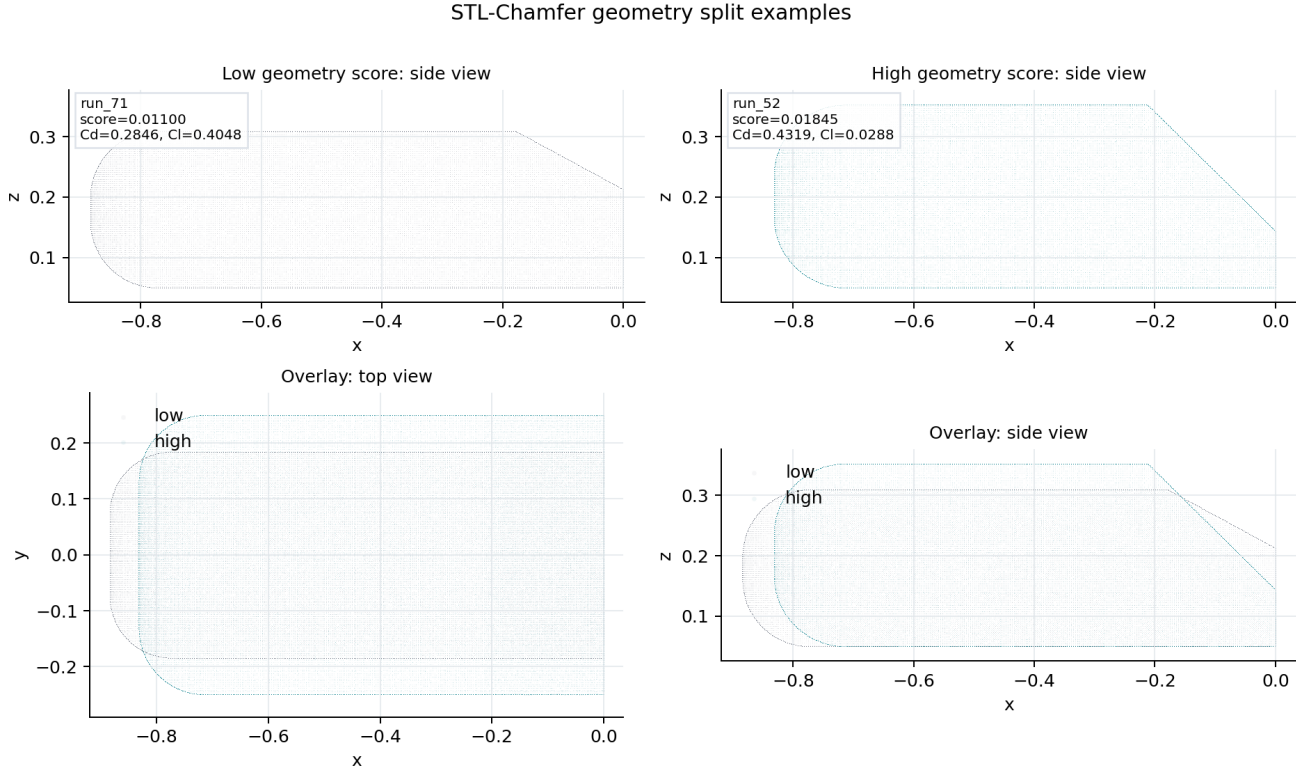


Figure 1: Geometry-score examples for `geometry`. The low-score case is `run_71` with Chamfer score 0.010996, $Cd=0.2846$, and $Cl=0.4048$. The high-score case is `run_52` with Chamfer score 0.018450, $Cd=0.4319$, and $Cl=0.0288$. The lower panels overlay the low-score and high-score STL vertex projections in a common coordinate frame.

high_drag and low_drag

Both splits sort all 500 cases by drag coefficient `cd`. `high_drag` holds out the top 20% by `cd`; `low_drag` holds out the bottom 20% by `cd`. In both cases, train/validation are sampled from the complementary 80% so validation remains in-distribution with training. Figure 3 shows these holdouts across run IDs.

In the current CSV, the highest-drag runs include `run_362`, `run_205`, `run_284`, `run_280`, and `run_315`. The lowest-drag runs include `run_390`, `run_238`, `run_122`, `run_117`, and `run_10`.

image_wake

The image-wake split is computed from downloaded `UxMean` PNGs under an external asset directory with `run_N/images/UxMean` subfolders. Each run contributes the Y-4 centreline/near-centreline side image plus three near-base cross-plane images: X-14, X-15, and X-16. For each image, the script scores the lower flow region using a reproducible color-intensity measure, then combines the Y-4 score with the mean near-base X score. The top 20% of

runs by this image score form the OOD test set, and validation is sampled from the complementary training-side pool. Figure 3 shows both the score used to define `image_wake` and the same split colored on drag coefficient.

In the current metrics, the highest image-wake scores include `run_280`, `run_205`, `run_284`, `run_309`, `run_315`, `run_362`, `run_171`, and `run_295`. Figure 2 compares the lowest and highest image-wake score cases using the Y-4 centreline image and a nearer-base X-14 image from the same PNG source used by the score.

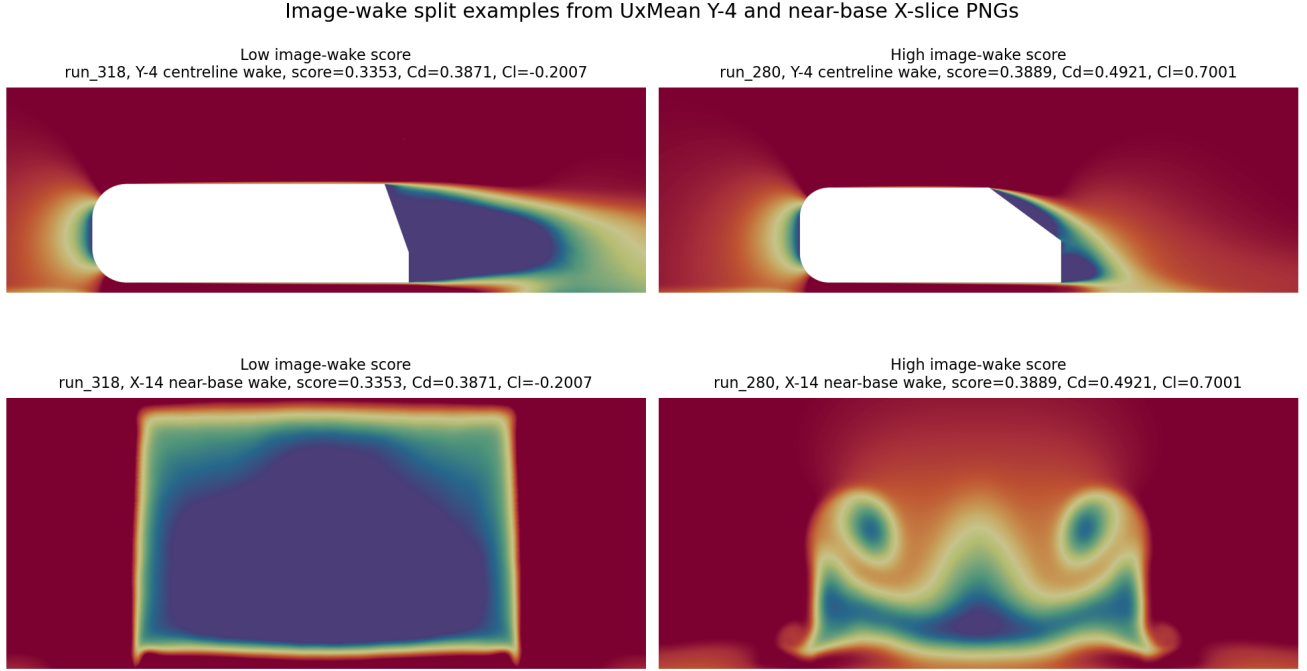


Figure 2: Image-wake examples for `image_wake`. The low-score case is `run_318` with image score 0.3353, Cd=0.3871, and Cl=-0.2007. The high-score case is `run_280` with image score 0.3889, Cd=0.4921, and Cl=0.7001. The top row shows Y-4, which captures the centreline wake, and the bottom row shows the nearer-base X-14 slice.

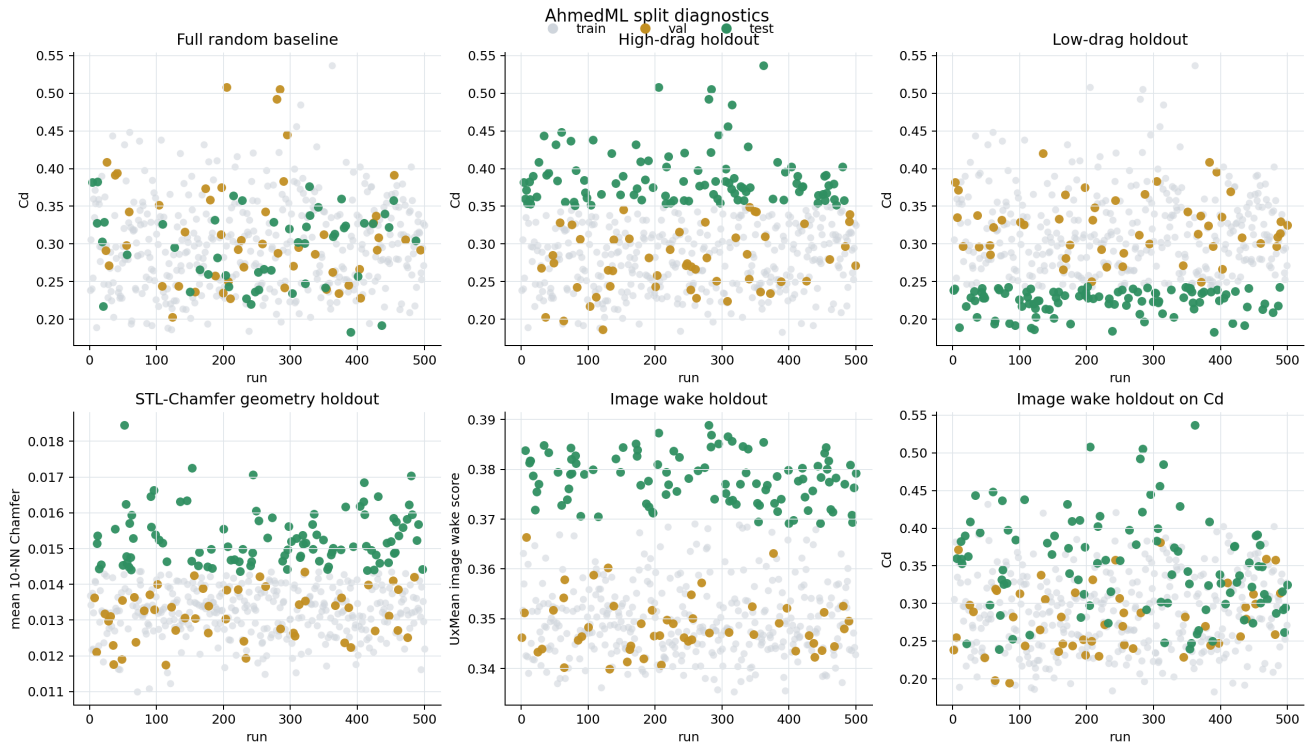


Figure 3: AhmedML split diagnostics. The panels show the full random baseline, high-drag holdout, low-drag holdout, STL-Chamfer geometry holdout, image-wake holdout, and image-wake split colored on Cd. The x-axis is run ID, and points are colored by each split’s train, validation, and test partitions.

Repeatability and transparency

The committed manifest is intended for normal use. The commands below are for auditing the split construction, recreating the figures, or refreshing the artifacts after changing the source data.

From a clean split-package checkout, download `force_mom_all.csv`, `geo_parameters_all.csv`, STLs, and the UxMean PNGs needed for the image-wake split into a directory outside this repository, then regenerate the metrics, manifest, and diagnostic plot:

```
ASSET_ROOT=../ahmedml_hf_assets
```

```
python3 scripts/download_hf_inputs.py --output-dir "$ASSET_ROOT" \
  --include-stls --include-wake-images --workers 6
python3 scripts/compute_chamfer_splits.py --data-root "$ASSET_ROOT" \
  --output-dir /tmp/ahmedml_chamfer_4096 --samples 4096 \
  --workers 16 --sample-workers 2 --runs all \
  --base-manifest splits/manifest.json
cp /tmp/ahmedml_chamfer_4096/chamfer_metrics.csv data/chamfer_metrics.csv
python3 scripts/compute_image_metrics.py --data-root "$ASSET_ROOT" \
  --output data/image_metrics.csv
AHMEDML_DATA_ROOT="$ASSET_ROOT" python3 scripts/generate_splits.py
python3 scripts/visualize_splits.py
python3 scripts/create_example_figures.py --asset-root "$ASSET_ROOT"
```

Full report regeneration then uses:

```
latexmk -pdf -cd docs/README.tex
```

For an existing dataset checkout, set `AHMEDML_DATA_ROOT` to the directory containing `force_mom_all.csv` and `geo_parameters_all.csv`. Set `AHMEDML_ASSET_ROOT`, or pass `-asset-root`, to the directory containing the `run_*/` STL and PNG assets. The repository's own `data/` directory should remain limited to small CSV files.

The committed source artifacts were generated against:

```
data/force_mom_all.csv
data/geo_parameters_all.csv
<asset-root>/run_*/ahmed_*.stl
<asset-root>/run_*/images/UxMean/*.png
data/chamfer_metrics.csv
data/image_metrics.csv
data/parameter_geometry_metrics.csv
docs/geometry_score_examples.png
docs/wake_score_examples.png
splits/manifest.json
```

Manifest format

```
{
  "full_train": ["run_1", "run_2", "..."],
  "full_val": ["run_24", "..."],
  "full_test": ["run_4", "..."],
  "scarce_train": ["run_10", "..."]
}
```

Case IDs match on-disk directory names and are sorted numerically by run number.

References

- [1] AhmedML dataset. <https://huggingface.co/datasets/neashton/ahmedml>.
- [2] AhmedML paper. <https://arxiv.org/abs/2407.20801>.
- [3] Noether AhmedML split. <https://github.com/Emmi-AI/noether/blob/main/src/noether/data/datasets/cfd/caeml/ahmedml/split.py>.